Category C Problem Statement

PROBLEM 1 • Easy

Hackathon Score Curve Booster

Problem Statement

The judging panel at a campus hackathon realizes the coding round was tougher than intended, and wants to give every team a flat bonus before the final leaderboard is printed. Write a method that boosts every score in place — no new array, no return value needed — and print the result using the standard library's own formatting helper.

Requirements

- curveScores(...) must modify the caller's original array directly — it should return nothing at all.
- Use Arrays.toString(...) to print the final leaderboard, rather than looping and printing manually.

Function Signature(s)

```
static void curveScores(int[] scores, int bonus)
```

Examples

Input	Output

`int[] scores = {70, 85, 60};`
`curveScores(scores, 10);`
`Arrays.toString(scores)`
	`"[80, 95, 70]"`

Explanation: *Each score is increased by 10 directly inside the array the caller already has — no reassignment needed back in main().*

HINT Remember: an array parameter gives the method direct access to the same boxes the caller has. You don't need to return anything for the caller to see the change.

CONSTRAINTS scores.length up to 200. bonus is a non-negative integer.

PROBLEM 2 • Easy

Duplicate Team Name Finder

Problem Statement

Hackathon registration is chaotic, and the organizers suspect the same team accidentally submitted their name twice under two different form entries. Write a method that scans the list of registered team names and reports the first duplicate it finds.

Requirements

- Compare every name against every other name using plain nested loops — no Collections class of any kind.
- Report the first duplicate found, scanning in order; if none exist, say so clearly.

Function Signature(s)

```
static String findDuplicateTeam(String[] teamNames)
```

Examples

Input	Output
`{"ByteForce", "CodeCrafters", "ByteForce"}`	`"Duplicate Found: ByteForce"`

Input	Output
{"ByteForce", "CodeCrafters", "NullPointers"}	"No Duplicates Found"

HINT For each name, only compare it against the names that come after it — comparing a pair twice (A vs B, then later B vs A) wastes effort without finding anything new.

CONSTRAINTS Up to 100 team names. Comparison is case-sensitive.

PROBLEM 3 • Intermediate

Top-3 Podium Finder

Problem Statement

With hundreds of teams submitting scores, sorting the entire array just to find the top 3 podium finishers is overkill — a judge on stage wants the podium announced the instant scoring wraps up, without waiting for a full sort. Find the top 3 scores in a single pass through the array.

Requirements

- Do not sort the array and do not use Arrays.sort(...) — track the top 3 scores as you scan once, left to right.
- Return the three scores in descending order.

Function Signature(s)

```
static int[] findTopThreeScores(int[] scores)
```

Examples

Input	Output
{45, 82, 79, 90, 33, 90, 61}	[90, 90, 82]

Explanation: *Two teams tied for the top score (90), so both appear — the third-place score is 82, not 79, since a genuine tie for first still only leaves one "true" second-highest slot for 82 to claim.*

HINT Keep three running variables — first, second, third — and update all three, in the right order, every time a new score beats one of them. Work out on paper which ones shift when a new highest score arrives.

CONSTRAINTS scores.length is at least 3, up to 10,000.

PROBLEM 4 • Intermediate

Hackathon Seating Grid Optimizer

Problem Statement

The venue is arranged as a grid of seating rows, and organizers want to walk the floor toward the rows where teams are visibly struggling (to offer mentor help) versus the rows that are humming along fine. Classify every row using one small, reusable helper method rather than repeating the averaging logic for each row by hand.

Requirements

- Write a private helper, rowAverage(int[] row), and call it once per row from the main method — do not recompute an average inline more than once.
- A row averaging below the threshold is a "Quiet Zone"; at or above it, a "Buzzing Zone".

Function Signature(s)

```
static double rowAverage(int[] row)
static String classifyRows(int[][] seatingScores, int threshold)
```

Examples

Input	Output
{ {40, 50, 45}, {85, 90, 95}, {30, 20, 25} }, threshold = 60	"Row 0: Quiet Zone | Row 1: Buzzing Zone | Row 2: Quiet Zone"

HINT The outer loop should be the only place that decides Quiet vs. Buzzing — rowAverage() should do nothing except compute one row's average and return it.

CONSTRAINTS Rows may vary in length (a jagged grid) — do not assume every row has the same number of columns.

PROBLEM 5 • Advanced

Placement Drive Shortlisting & Ranking Engine

Topics Integrated: *Arrays, Method Overloading, Static Methods, Standard Library (Arrays.sort), Constructors & Encapsulation*

Problem Statement

The placement cell wants one script to do two jobs: decide who's eligible for a company's coding round, and rank everyone who qualifies. Eligibility isn't one simple rule — a strong CGPA alone is enough, but a borderline CGPA can still qualify through a genuinely good coding-test score. Once shortlisted, candidates should be ranked by a composite score, using Java's own sort rather than writing a sort by hand.

Requirements

- Provide two overloaded isEligible(...) checks: a CGPA-only quick filter, and a combined CGPA-and-coding-score filter for borderline cases — decide the exact thresholds yourself.
- Candidate must implement Comparable<Candidate> so that Arrays.sort(...) alone can rank a shortlisted array by composite score, descending, with no separate comparison logic written elsewhere.

Function Signature(s)

```
public Candidate(String name, double cgpa, int codingScore)
static boolean isEligible(double cgpa)
static boolean isEligible(double cgpa, int codingScore)
int compareTo(Candidate other)
static String shortlistAndRank(Candidate[] candidates)
```

Examples

Input	Output
`shortlistAndRank({ new Candidate("Aisha", 8.2, 40), new Candidate("Rohit", 6.8, 65), new Candidate("Meena", 6.0, 90), new Candidate("Karan", 7.5, 20) })`	`"1. Aisha (102.0) | 2. Rohit (100.5) | 3. Karan (85.0)"`

Explanation: *Aisha and Karan clear the CGPA-only bar directly. Rohit's CGPA alone falls just short, but a strong coding score of 65 clears the combined check. Meena clears neither check and is left out entirely, regardless of her high coding score.*

HINT Once every shortlisted Candidate correctly implements compareTo(...), a plain Arrays.sort(shortlistedArray) does the entire ranking for you — resist the urge to write your own sort loop here.

CONSTRAINTS cgpa is 0–10. codingScore is 0–100. Up to 500 candidates in a batch.